智能·协同·信创
仓颉+泛微 构建智慧公文平台

泛微网络 晏成

01

泛微公司简介

数字泛微，协同办公行业的领军企业



志达宇泛、见著于微

泛微网络科技股份有限公司成立于2001年，总部设立于上海，专注于协同管理软件领域，帮助组织构建统一的数字化运营平台。泛微是**“国家规划布局内重点软件企业”，上交所主板上市公司（股票代码：603039）。**

1 个目标
为客户提供好用的协同管理软件

23 年
23年来始终专注协同管理软件的研发与服务

87 个行业
87个细分行业解决方案适应大部分企业组织

200 多家机构
200家本地化机构打造2小时内服务能力

10000 多位同仁
全国10000多名专业人士为客户提供全方位服务

80,000 多家客户
全国拥有80000多家中大组织客户，覆盖87个细分行业

组织规模	服务地域	销售业绩	客户总量
领先	领先	领先	领先

泛微 weaver 数字化办公

众多政务客户实践

外交部

公安部

国家税务总局

应急管理部

中共中央宣传部

中国人民银行

省市政府

中国共产党浙江省委
中国共产党江西省委
中国共产党贵州省委
中国共产党西藏自治区委员会
陕西省人民政府
吉林省人民政府
湖北省人民政府
宁夏回族自治区人民政府
西藏自治区人民政府
广州市人民政府
... ...

无锡市人民政府
淄博市人民政府
大连市人民政府
乐山市人民政府
南昌市人民政府
岳阳市人民政府
荆门市人民政府
定西市人民政府
银川市人民政府
保定市人民政府
... ...

区县政府

天津市武清区人民政府
天津市宁河区人民政府
重庆市綦江区人民政府
重庆市大足区人民政府
广州市海珠区人民政府
广州市荔湾区人民政府
石家庄市裕华区人民政府
廊坊市广阳区人民政府
保定市竞秀区人民政府
大连市普兰店区人民政府
... ...

宁波市奉化区人民政府
潍坊市潍城区人民政府
武汉市汉阳区人民政府
株洲市渌口区政府
岳阳市云溪区人民政府
长沙市天心区人民政府
桂林市象山区人民政府
南宁市兴宁区人民政府
西安市鄠邑区人民政府
汉中市汉台区人民政府
... ...

厅局委办

吉林省教育厅
宁夏回族自治区教育厅
山西省教育厅
辽宁省教育厅
陕西省教育厅
内蒙古自治区公安厅
北京市公安局
上海市公安局
佛山市公安局
湖北省财政厅
... ...

上海市发展和改革委员会
江苏省发展和改革委员会
重庆市发展和改革委员会
云南省发展和改革委员会
上海市大数据中心
大连市大数据中心
拉萨市大数据发展管理局
绍兴市大数据发展管理局
河北省卫生健康委员会
陕西省卫生健康委员会
... ...

事业单位

财政部政府和社会资本合作中心
工业和信息化部产业发展促进中心
国家海洋局南海环境监测中心
公安部鉴定中心
中国疾病预防控制中心
中国生物技术发展中心
中国版权保护中心
国家密码管理局商用密码检测中心
中国计量科学研究院
中国科学院地理科学与资源研究所
... ...

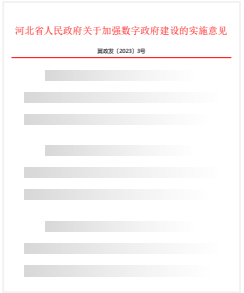
众多的央国企客户实践



泛微 weaver 数字化办公

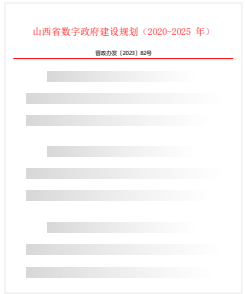
公文管理仍是各地政府及国央企协同办公关注的核心

数据跨组织贯通与AI与办公场景融合，是目前党政机关和国央企关注的核心需求



《河北省人民政府关于加强数字政府建设的实施意见》

提升全省**公文"一张网"**运转水平。加快实现县级非涉密移动办公，并逐步推广到乡(镇、街道)。建立非涉密移动办公**标准化数据接口**，实现省、市、县三级贯通。推动涉密移动办公向省重点部门和各市、雄安新区延伸。



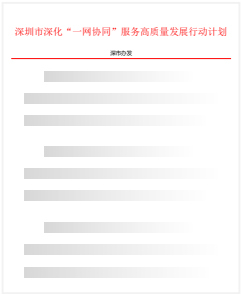
《山西省数字政府建设规划（2020-2025 年）》

在全省范围构建协同联动、集约高效的“PC端+移动端”一体化办公平台。制定**电子公文交换标准**,通过电子印章及区块链技术,建 设**全省统一、安全可信**的电子公文交换系统。



《黑龙江省“十四五”数字政府建设规划》

围绕日常办公需求，提供 **OCR 文字识别、语音识别、文字转译、在线文档、手写签批等通用服务**，提高“指尖”办文、办会、办事水平。推动政府部门内部非涉密公文电子化流转，逐步减少纸质公文，形成**线上闭环，助力跨地区、跨层级、跨部门业务协同。**



《深圳市深化“一网协同”服务高质量发展行动计划》

提升**电子公文标准化水平**，规范电子文件的全生命周期管理。推动**政务智能(GI)应用**，基于“文会事”基础业务进行分析研究，探索不同业务数据语义相通，支撑跨业务共享协同。强化平台分析和综合研判能力，服务领导决策。

泛微新一代信创智慧公文平台

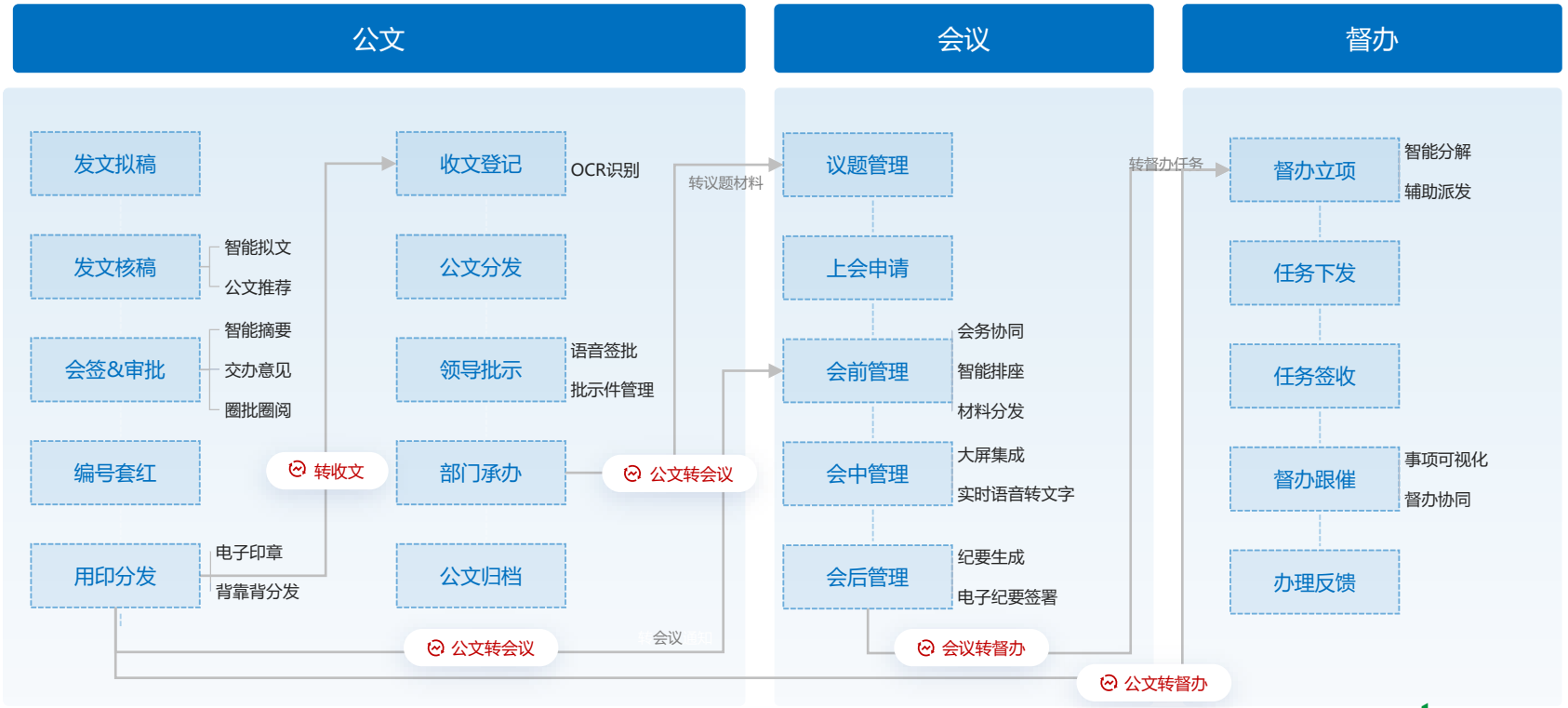
泛微通过仓颉，重构了公文管理系统，为客户提供智能、协同、信创的智慧公文平台



02

智慧公文平台的亮点

智慧公文平台，文事会一体化业务模型

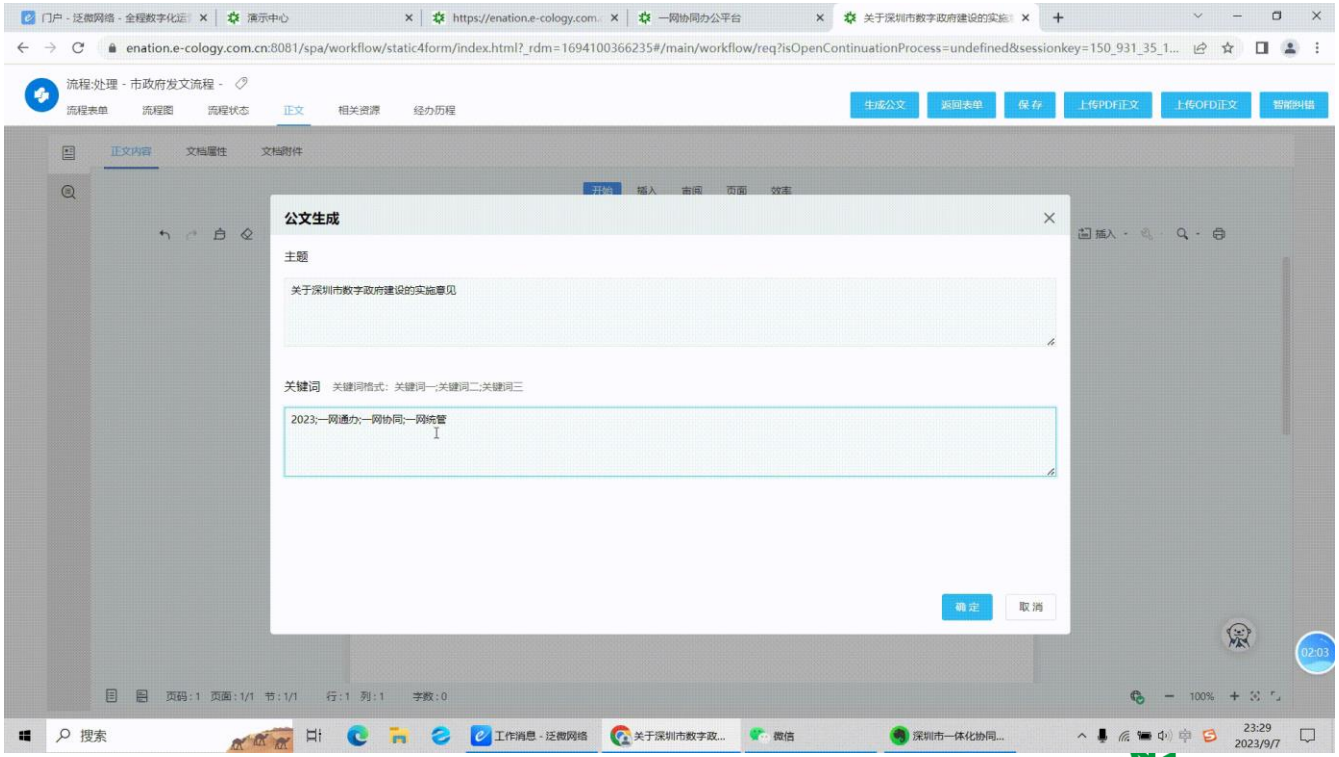


泛微 weaver 数字化办公

发文管理，AI智能拟文

AI智能拟文

只要输入标题和内容，迅速获取用户的真实写作意图，在电子公文资源库比对历史数据中的各种公文材料，自动识别出与输入内容关联、事项特征吻合的文件资料，自动生成范文，范文带有标准的公文格式

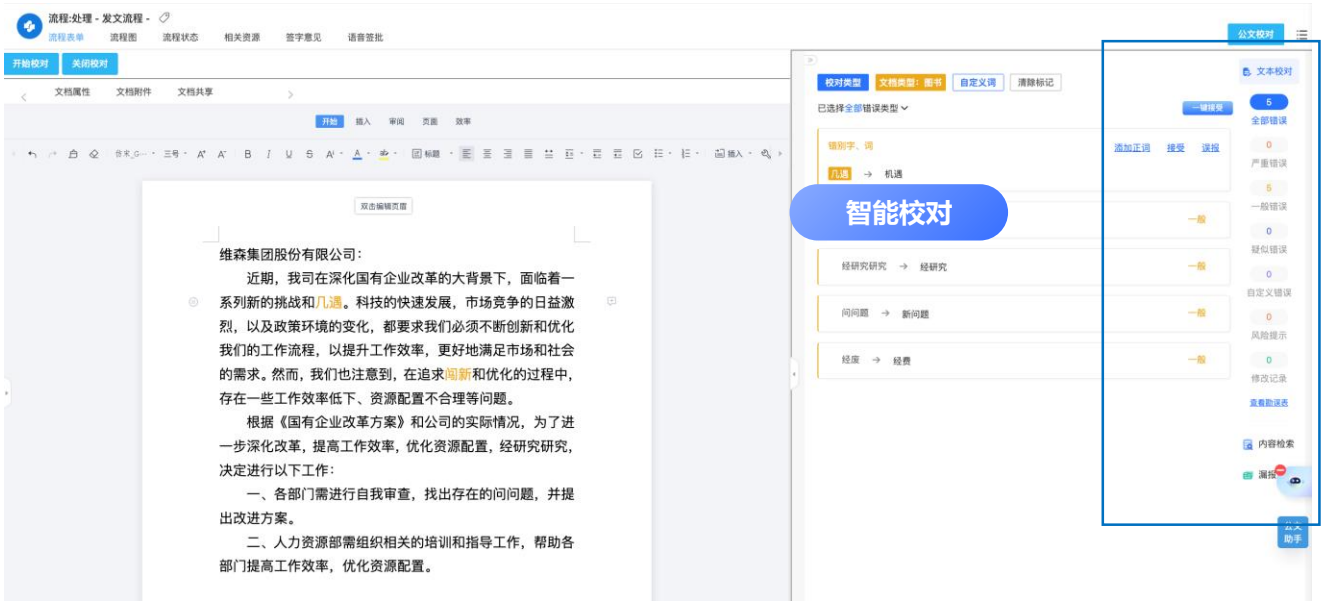


泛微 weaver 数字化办公

发文管理，公文智能校编

公文智能校编

对文种选用、格式规范、语言文字、语法规范、标点符号等错误进行纠错，对敏感词/句、政治表述进行内容校验；从原先以小时消耗计时的工作可降低至秒级 。



发文管理，公文智能摘要

公文智能摘要

智能摘要能力，从原先需要通篇阅读的公文智能识别，帮助办公室主任精确获知公文内容，减少阅文时间

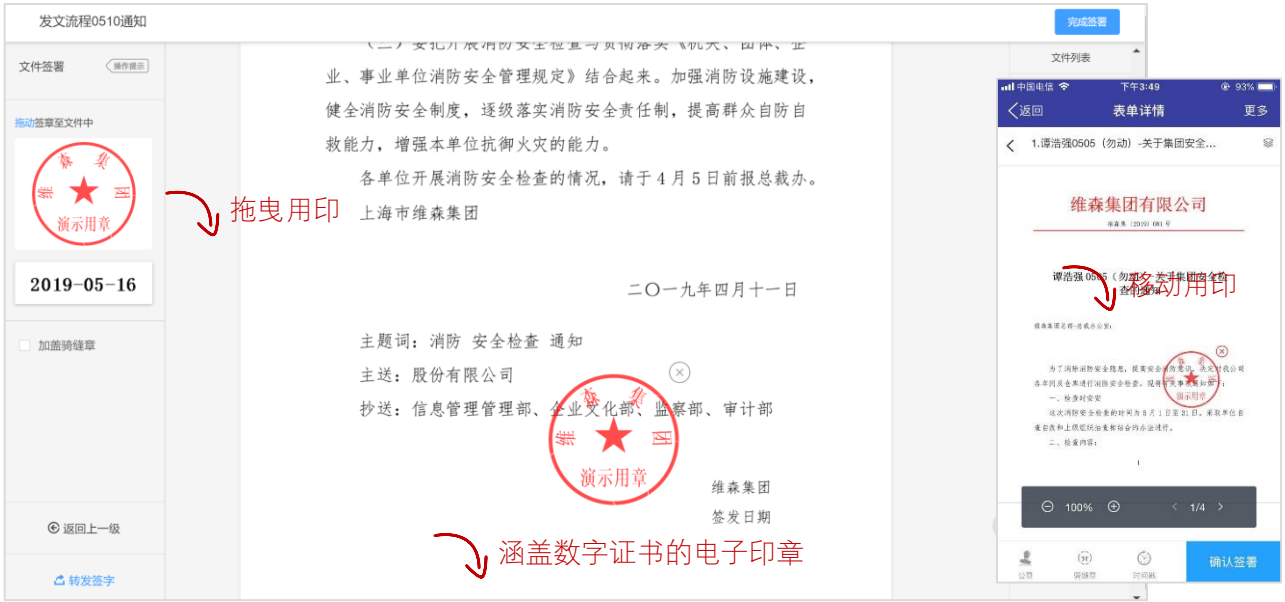


泛微 weaver 数字化办公

发文管理，公文电子印章

公文电子印章

融合数字签名技术，拖拉拽用印，支持联合发文、联合用印；严格用印权限控制， 支持公文移动电子用印。



泛微 weaver 数字化办公

收文办理，OCR智能收文

OCR智能收文

通过拍照OCR识别或上传电子文件（WORD、WPS、PDF、OFD等）就可以自动登记公文，自动识别标题、文号等关键信息，智能化完成登记。



泛微 weaver 数字化办公

收文办理，移动语音办文

发文管理　收文办理　会议决策　督办落实

移动语音办文

通过领导通过手机，说句话即可批示公文，批示内容可在公文内容中任意标记，并支持语音转文字功能，方便秘书对领导批示进行抄清、代录



自然语言识别



抄清、代录

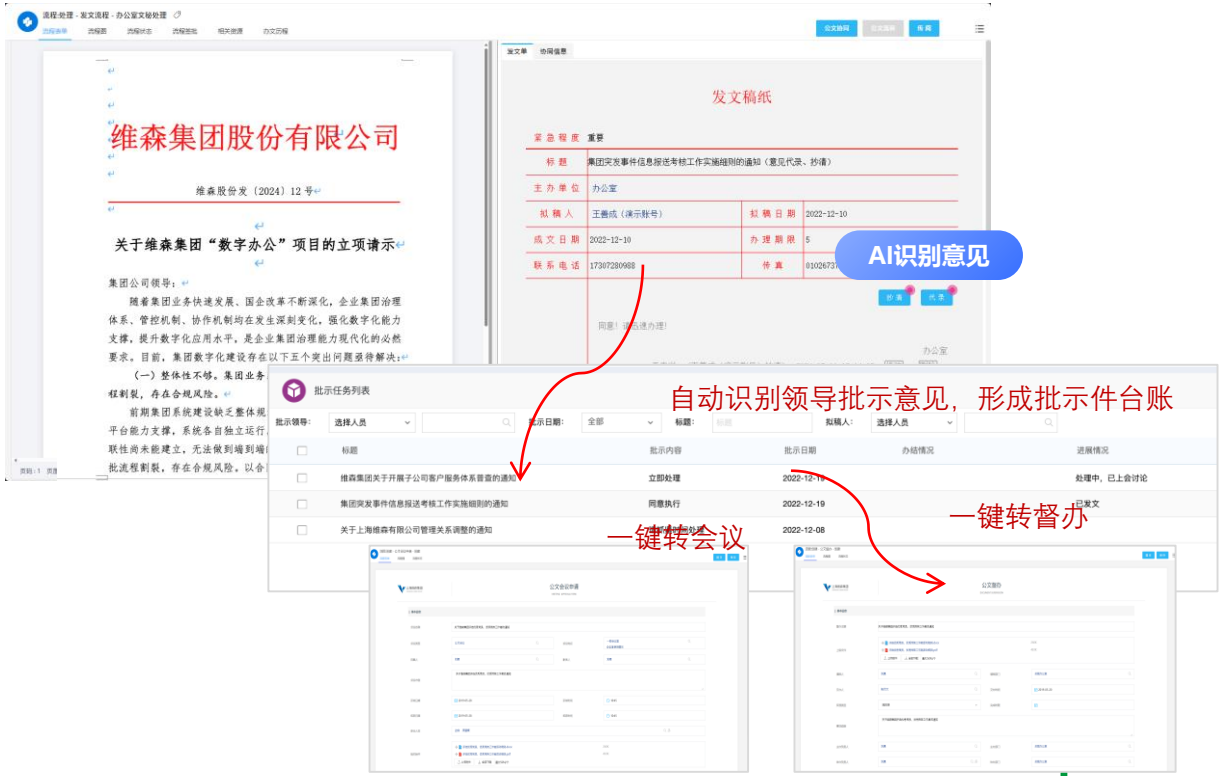
泛微 weaver 数字化办公

收文办理，智能批示分办

发文管理　收文办理　会议决策　督办落实

智能批示分办

对领导的非常规批示意见进行智能识别，判断领导意图，并提醒办公室主任跟进办理过程，支持一键转督办、一键转会议 。



会议决策，智能语音速记

发文管理　收文办理　会议决策　督办落实

会议语音速记

视频会议、语音笔记等，可实时转换会议语音形成会议文字纪要

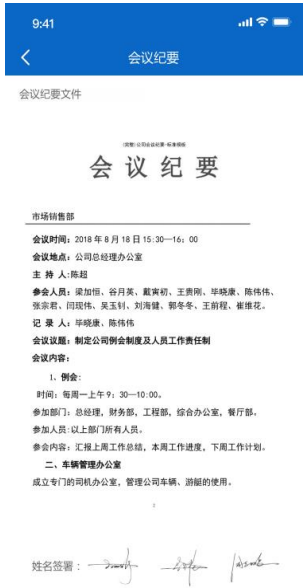
根据会议每个领导讲话的可识别每个领导的身份，做到语音转文字记录角色分离

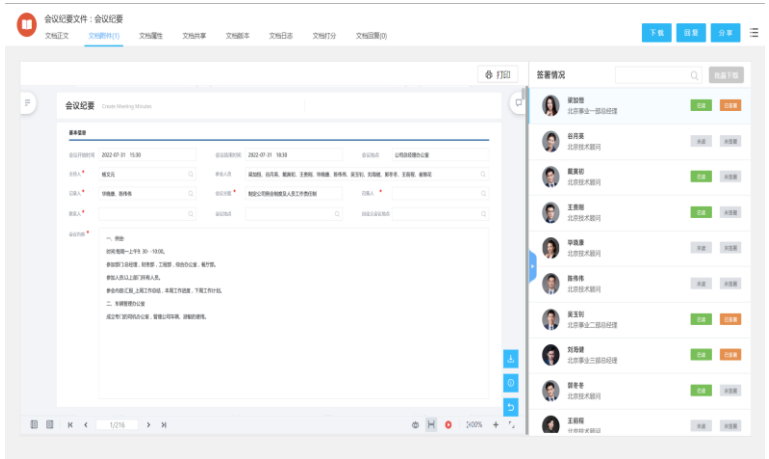


会议决策，电子纪要签署

电子纪要签署

会议过程形成的会议纪要、决议等重要文件，可对接电子签平台进行电子签署，签署文件具备相关电子签名，防篡改





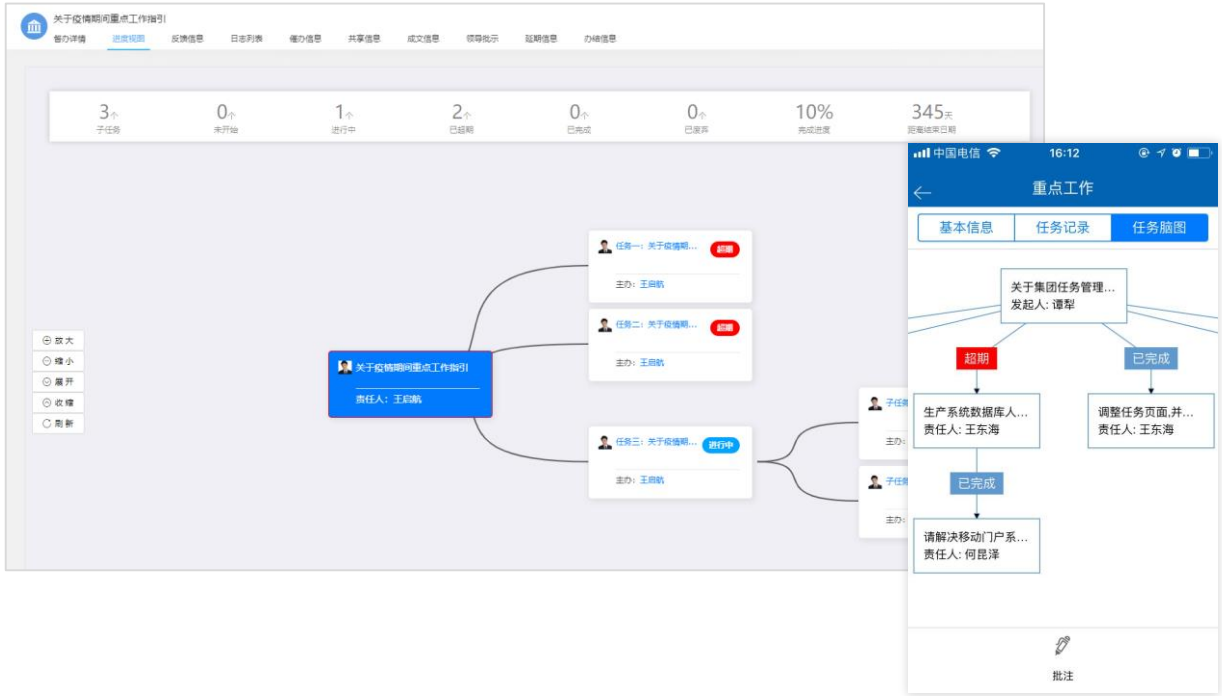
泛微 weaver 数字化办公

督办落实，智能任务分解

智能任务分解

支持公文审批过程中，一键生成督办任务，并对公文内容智能识别，分解进行督办任务。

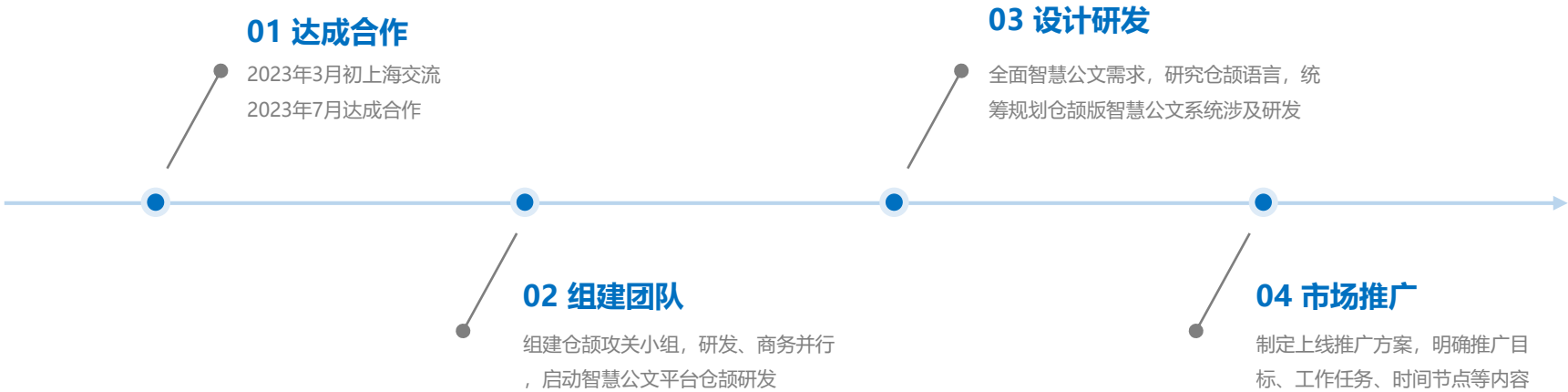
通过对文本内容的自动分析，系统自动生成相应任务事项进行派单



03

为什么与仓颉合作

泛微同华为仓颉合作历程



完全基于信创技术路线的智慧公文平台

仓颉亮点

全链路国产化

服务端：已全面满足主流的信创生态，尤其是华为本身的生态

终端：开发应用全面适配主流信创终端，如鸿蒙、麒麟、统信等终端操作系统

应用：仓颉本身就是100%国产化的语言

信创终端

操作系统

- UOS
- 麒麟

浏览器

- 火狐
- 360浏览器
- UOS浏览器

非信创终端

操作系统

- Windows
- macOS
- Linux

浏览器

- 火狐
- 360浏览器
- IE
- Chrome

信创服务端

CPU

- 龙芯、飞腾、鲲鹏、...

数据库

- 达梦、人大金仓、Openguass...

操作系统

- 麒麟系列、统信UOS、欧拉

中间件

- 东方通、中创、金蝶天燕、宝兰德

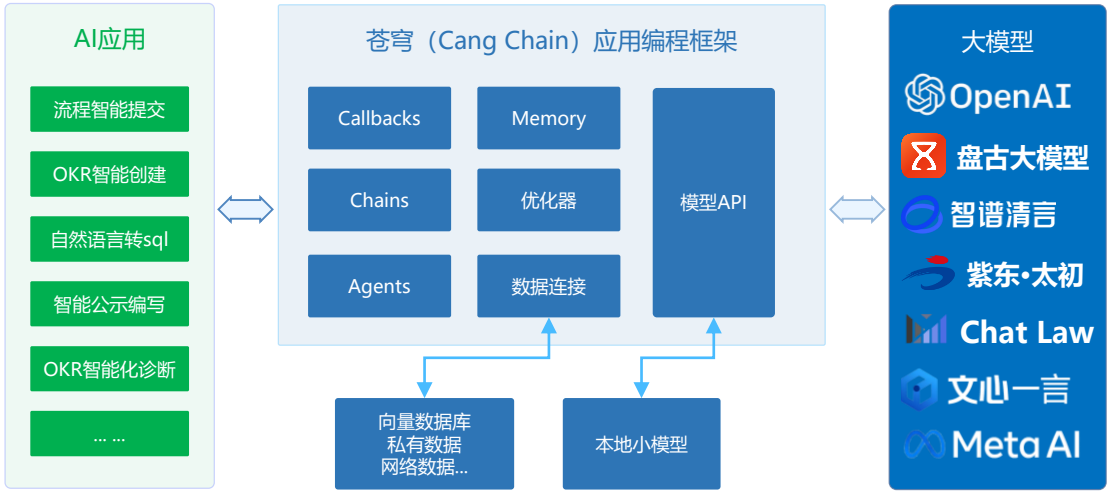
泛微 weaver 数字化办公

整合主流大模型API或SDK，快速融入应用场景

仓颉亮点

AI生态能力

提供成熟的AI应用变成框架，无需在自定搭建AI的开发环境，多种通用模型、专业模型的SDK、API接口可按需调用，构建多个智能化场景



开发工具、组件完善，极大提升开发人员效率

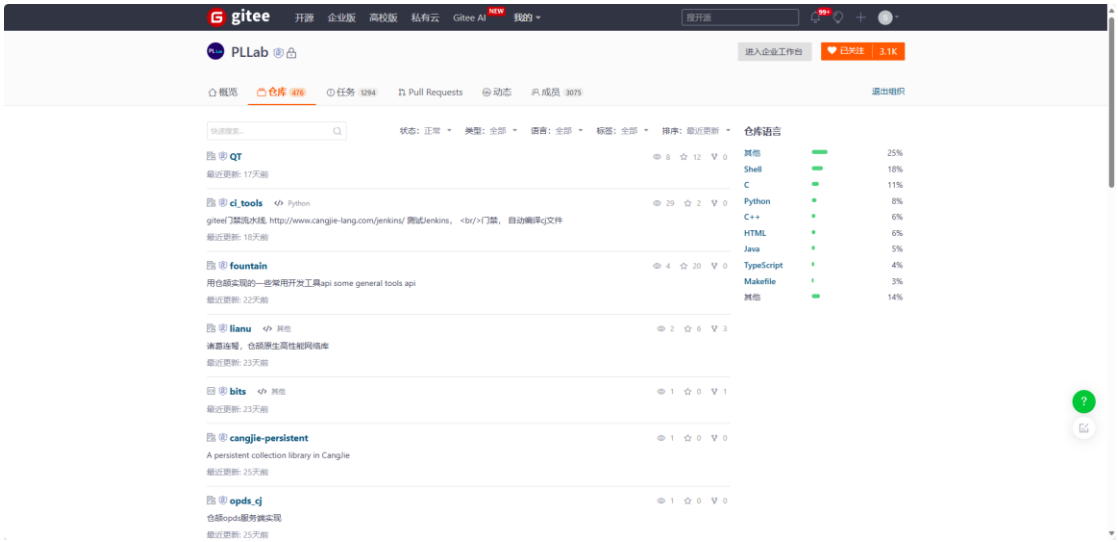
仓颉亮点

开发效率高

具备成熟的代码库以及可复用的代码片段

提供从开发工具、编译工具、测试工具完整的开发生态

已适配主流的软硬件技术栈，无需单独适配开发



运行效率相较传统开发应用，提高3-5倍

仓颉亮点

应用运行快

经过压力测试，在基于mysql和openguass的技术路线下；

整体性能相较于传统开发应用，整体提高3-5倍

仓颉已完成对访问层、数据库连接层以及数据库本身的性能优化

应用服务器（2台）：
CPU：16核（鲲鹏）
内存：32G
操作系统：麒麟V10

海量（Openguass） 数据库服务器（1台）：
CPU：64核（鲲鹏）
内存：256G
操作系统：麒麟V10

5.1. 混合场景测试汇总结果

用例名称	事务名称	20 并发		50 并发		70 并发	
		平均响应时间（ms）	TPS(笔/秒)	平均响应时间（ms）	TPS(笔/秒)	平均响应时间（ms）	TPS(笔/秒)
待办公文	待办公文	316	1.00003	419	2.2998	962	2.73668
已办公文	已办公文	1029	0.75957	1186	1.78188	1310	2.16611
待办公文查看	待办公文查看	1093	0.53888	1212	1.20377	1437	1.36246
公文创建审批	新建公文-创建	2153	0.33125	2600	0.75902	2874	1.02179
	新建公文-审批提交	1952	0.33157	2543	0.75884	2438	1.02133
公文签收	公文签收	3264	0.3246	2737	0.74789	2732	1.01671
公文分发	公文分发	37672	0.04809	40492	0.11311	43131	0.14875
TPS 合计			3.34		7.66		9.47

用例名称	事务名称	100 并发		130 并发		150 并发	
		平均响应时间（ms）	TPS(笔/秒)	平均响应时间（ms）	TPS(笔/秒)	平均响应时间（ms）	TPS(笔/秒)
待办公文	待办公文	1815	3.04073	2283	3.04666	2690	3
已办公文	已办公文	1698	2.60554	3723	2.29864	4566	1.96701
待办公文查看	待办公文查看	1614	1.56047	2988	1.32097	3775	1.3388
公文创建审批	新建公文-创建	3614	1.255	7338	0.99172	10048	0.85387
	新建公文-审批提交	2997	1.25561	6108	0.97705	8226	0.84707
公文签收	公文签收	3649	1.2428	7566	0.93700	12440	0.73566
公文分发	公文分发	48796	0.18557	66133	0.18063	86124	0.16338
TPS 合计			11.14		9.74		8.90

备注：响应时间是内网后端测试时间，不包括网络延时和前端页面加载、渲染时间；oa 请求为异步处理故选取最耗时的请求响应时间作为统计的事务响应时间。

服务器资源统计：

服务器	并发数	CPU 使用率	内存使用率
应用服务器	20	30%-40%左右	45%50%左右
数据库服务器		15%-20%左右	55%-60%左右
应用服务器	50	40%-45%左右	45%50%左右
数据库服务器		25%-30%左右	55%-60%左右
应用服务器	70	45%-50%左右	45%50%左右
数据库服务器		35%-40%左右	55%-60%左右
应用服务器	100	50%-55%左右	45%50%左右
数据库服务器		45%-50%左右	55%-60%左右
应用服务器	130	55%-60%左右	45%50%左右
数据库服务器		70%左右	55%-60%左右
应用服务器	150	60%-65%左右	45%50%左右
数据库服务器		75%左右	55%-60%左右

泛微 weaver 数字化办公

确保开发标准一致，保障代码安全和质量

仓颉亮点

元编程技术

通过元编程技术，解决了代码的标准及规范问题

- **安全性**：编译阶段自动进行安全性检查，尤其是一些明显的安全隐患、外部方法等，自动检查调用
- **合规性**：编译阶段自动进行合规检查，给出代码修改建议
- **自动续写**：只需要写出对应方法，元编程自动会帮助我们补充完后续代码



CHAPTER 13

元编程

元编程是一种将计算机程序（代码）当做数据的编程技术，从而修改、更新、替换已有的程序。例如可以将一些计算过程从运行时挪到编译时，并在编译期进行代码生成。仓颉语言提供的元编程能力，能支持代码复用、操作语法树，编译期求值，甚至自定义文法等功能。

下面是一个利用元编程解决具体问题的例子，利用仓颉宏为某些需要递归计算的函数进行记忆优化。

```
// macro_definition.cj
macro package memory

from std import ast.*

func checkBooleanAttr(attr: Tokens): Bool {
    // true or false
    if (attr.size != 1 || attr[0].kind != TokenKind.BOOL_LITERAL) {
        throw IllegalArgumentException("Attribute for memoize should be true or false")
    }
    return attr[0].value == "true"
}

public macro memoize(attr: Tokens, input: Tokens): Tokens {
    let memoized: Bool = checkBooleanAttr(attr)

    // no memorization
    if (!memoized) {
        return input
    }
```

(接下页)



13.2 宏

仓颉实现元编程的主要方式是使用宏。仓颉的宏是语法宏，其定义形式上类似于函数，也和函数一样可以被调用。不同点是：

1. 宏定义所在的 package 需使用 macro package 来声明。
2. 宏定义需要使用关键字 macro。
3. 宏定义的输入和输出类型必须是 Tokens。
4. 宏调用需要使用 @。

从输入代码序列到输出新的代码序列的这个映射过程称为宏展开。宏在仓颉代码编译时进行展开，一直展开到目标代码中没有宏为止。宏展开的过程会实际执行宏定义体，即宏是在编译期完成求值。展开后的结果重新作用于仓颉的语法树，继续后面的编译和执行流程。

13.2.1 macro package

仓颉宏的定义需要放在由 macro package 声明的包中，被 macro package 限定的包仅允许宏定义对外可见，其他声明包内可见。宏定义允许返回空 Tokens 对象，不包含任何代码（Tokens），但是如果返回空 Tokens 对象的）宏调用是其他表达式的一部分，编译将报错，因为空的 Tokens 对象无法构成有效的表达式。

- 示例

```
// file define.cj
macro package define          // 编译 define.cjo 需带 macro 属性
from std import ast.*
public func A() {             // func A 外部不可见，此处需报错
public macro M(input: Tokens): Tokens { // macro M 外部可见
  return input
}
```

需要特殊说明的是，在 macro package 中仅允许 macro package 被重导出，在非 macro package 中仅允许非 macro package 被重导出。

- 示例

```
// A.cj
macro package A
from std import ast.*

public macro M1(input: Tokens): Tokens {
    return input
}

// B.cj
package B
public import A.* // error: it is not allowed to re-export a macro package in a...
```

(接下页)

版权所有，禁止扩散

泛微 weaver 数字化办公 主板上市公司 · 腾讯战略投资

泛微 weaver 数字化办公
主板上市公司 · 腾讯战略投资

智能 · 协同 · 信创

仓颉+泛微 共筑开发新模式